# Lecture 9
# I/O Devices

Mingyu Gao
gaomy@tsinghua.edu.cn

*IIIS, Tsinghua*

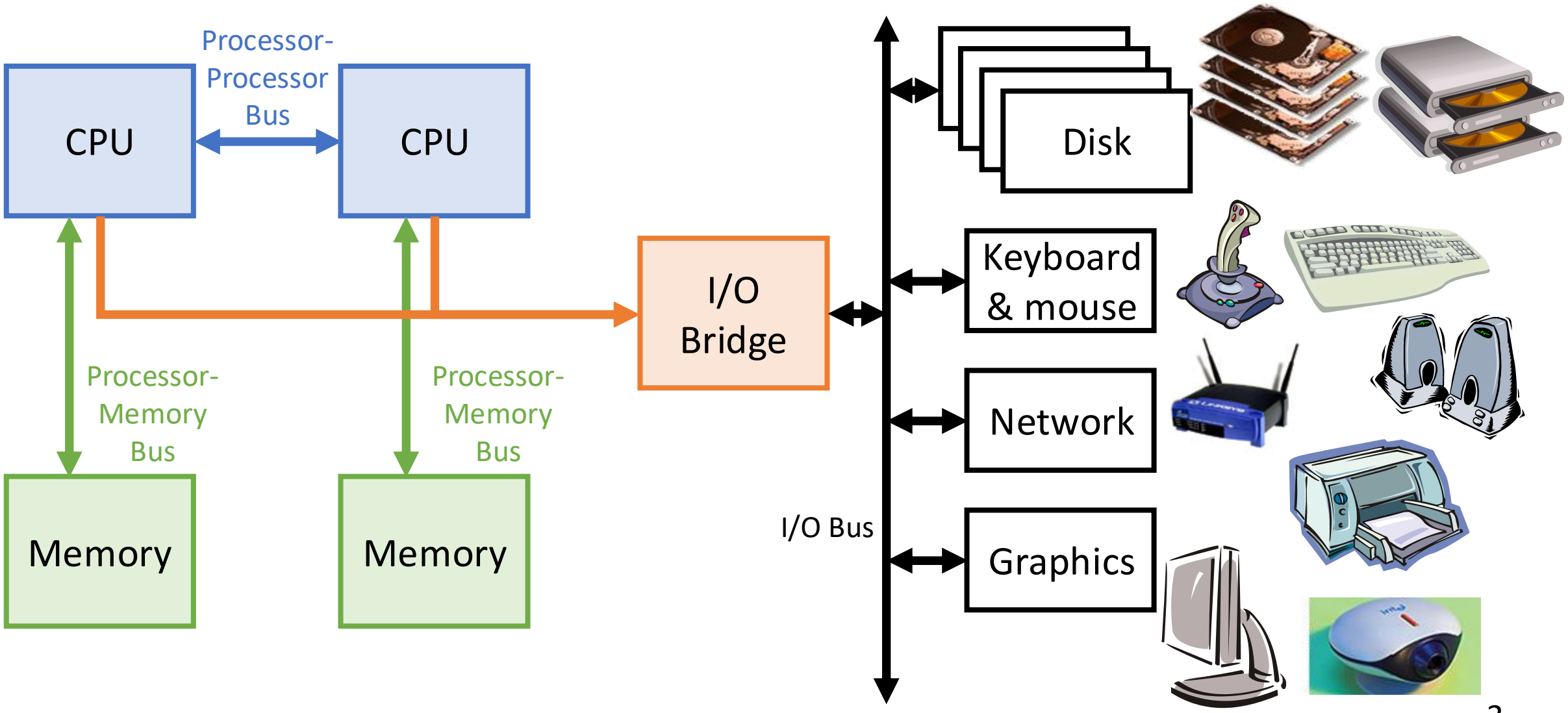
# Today's Focus – Input/Output (I/O)

- Without I/O, computers are useless (disembodied brains?)

- I/O challenge 1: thousands of devices, different from each other
  - In contrast to one piece of physical memory
- Need new and standardized interface and mechanisms

- I/O challenge 2: devices are slow and with unpredictable behaviors
  - Would stall processors for too long and waste resource
- Need new access approaches different from memory loads/stores

# I/O Devices in Computer Systems



# I/O Devices Diversity

- I/O devices can be characterized by
  - Behavior: input, output, communication, storage
  - Partner: human or machine
  - Data rate (bytes/sec, transfers/sec) and required response time
  - ……

| Device | Behavior | Partner | Data Rate (MB/s) |
|---|---|---|---|
| Keyboard | Input | Human | 0.00001 |
| Mouse | Input | Human | 0.02 |
| Laser Printer | Output | Human | 0.1 |
| Graphics | Output | Human | 5000.0 |
| Network | Communication | Machine | 1000.0 |
| Magnetic disk | Storage | Machine | 600.0 |
| Optical disk | Storage | Machine | 10.0 |

# Operational Parameters for I/O

- Data granularity: byte vs. block
  - Some devices provide single byte at a time (e.g., keyboard)
  - Others provide whole blocks (e.g., disks, networks)

- Access pattern: sequential vs. random
  - Some devices must be accessed sequentially (e.g., tape, networks)
  - Others can be accessed randomly (e.g., disks)

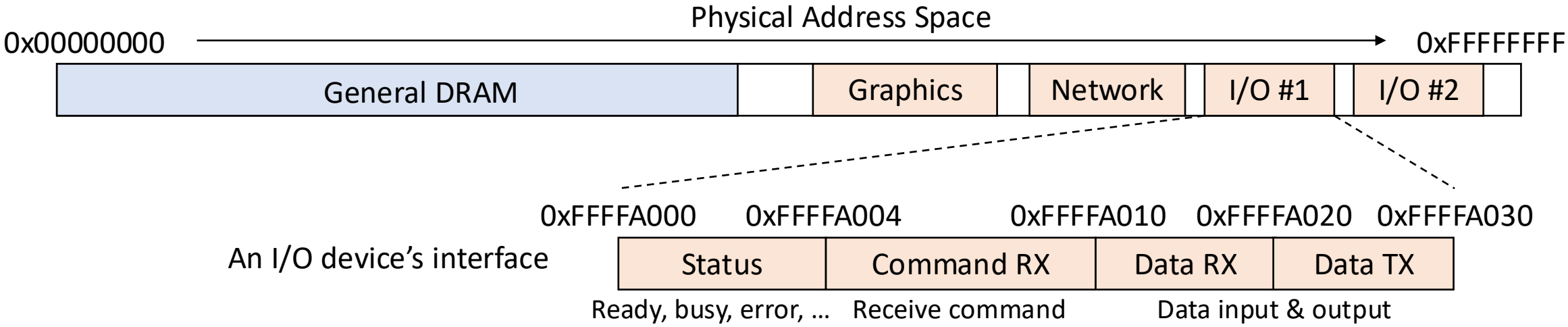
- Device speed rates vary over many orders of magnitude
  - System better be able to handle this wide range
  - Better not have high overhead per byte for fast devices
  - Better not waste time waiting for slow devices

# I/O Techniques

- How to access I/O devices?
  - Memory-mapped I/O
  - Standard interface and device drivers
- How to reduce long wait time for slow I/O devices?
  - I/O notification: polling and interrupts
- How to reduce access cost to slow I/O devices?
  - DMA

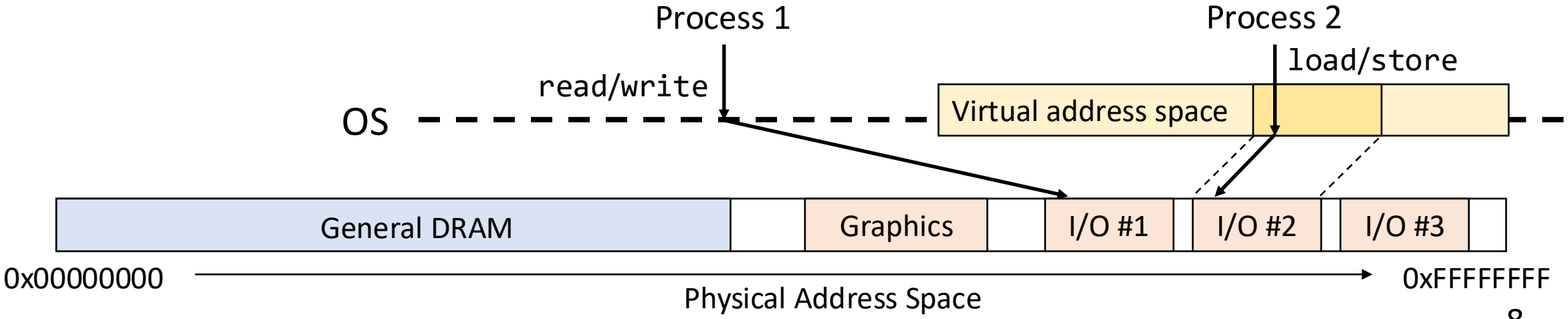
# Memory-Mapped I/O

- How to access I/O devices?
  - Each I/O device may have interface registers and data buffers, e.g., command, status, data input/output, …, determined by the device vendor
- Memory-mapped I/O: assign an unused range of physical address to each I/O device, which is called its I/O address
  - Read/write to these addresses is to read/write the interface registers/buffers, i.e., to control and communicate with the I/O device



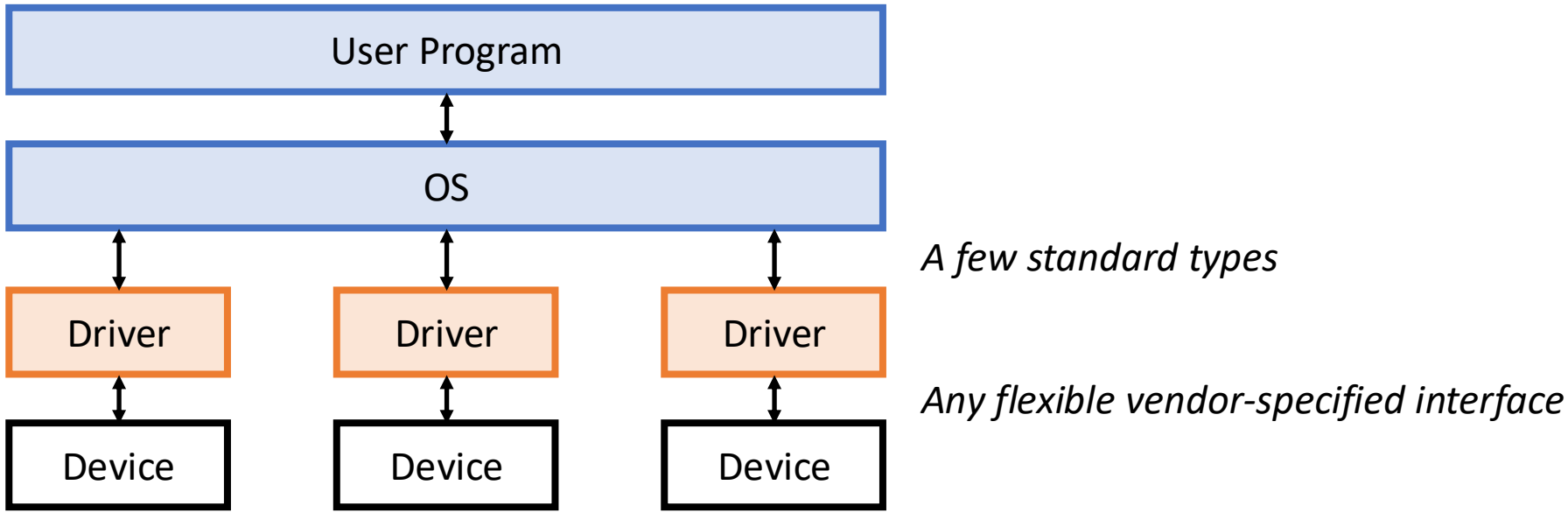
# Memory-Mapped I/O

- I/O address space is protected by the virtual memory mechanism
  - User programs are prevented form issuing I/O operations directly
- Method 1: use syscalls to access I/O addresses, e.g., `read`, `write`
  - OS will internally translate the access to an I/O device access
- Method 2: ask OS to map I/O addresses to virtual addresses, e.g., `mmap`
  - After mapping, can access I/O devices through normal load and store instructions, just like accessing normal virtual memory addresses



# Standardizing I/O Interface

- Goal: provide uniform interfaces for wide range of different devices
  - Recall that each device could have different ports and I/O address mappings
- Abstract all I/O devices to fit in several standard types
- Device driver, implemented by device vendors
  - Expose the standard abstract interface to the OS
  - Internally translate to low-level actions to perform on the device



# I/O Types By Access Granularity/Pattern

- Block device (e.g., disks, SSDs)
  - Access blocks of data; can randomly address a location (i.e., seek)
  - Commands include `open()`, `close()`, `read()`, `write()`, `seek()`

- Stream device
  - Sequential interface, no addressing; just read/write next block or character/byte
  - Character/byte device (e.g., keyboards, mice, serial ports)
    - Single byte at a time; commands include `get()`, `put()`
  - Network device (e.g., Ethernet, wireless, Bluetooth)
    - Different enough and critical enough to have their own interface, e.g., sockets
    - More details of networking in later lectures

# I/O Types By Timing

- Blocking interface, i.e., "wait"
  - When requesting data (e.g., `read()`), put process to sleep until data is ready
  - When sending data (e.g., `write()`), put process to sleep until device is done

- Non-blocking interface, i.e., "do not wait"
  - Returns quickly from read or write, with count of bytes successfully transferred
  - Only best-effort; count of bytes may be less than the request, or even zero

- Asynchronous interface, i.e., "tell me later"
  - Provide user's buffer (to be read/write), return immediately, do actual work in background in OS kernel (fills in buffer or takes data), and notify user when done

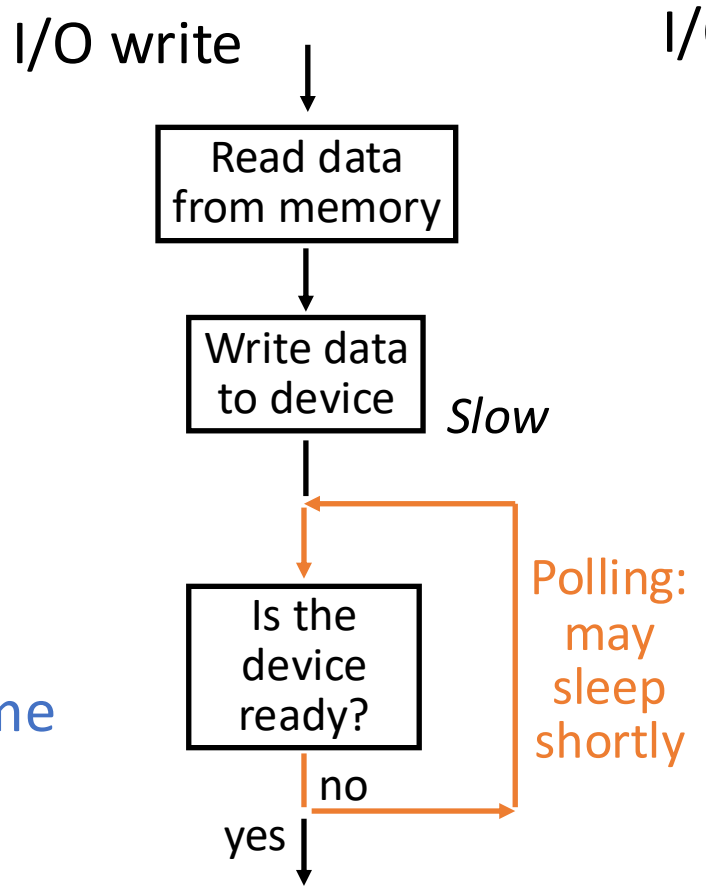
# I/O Notification

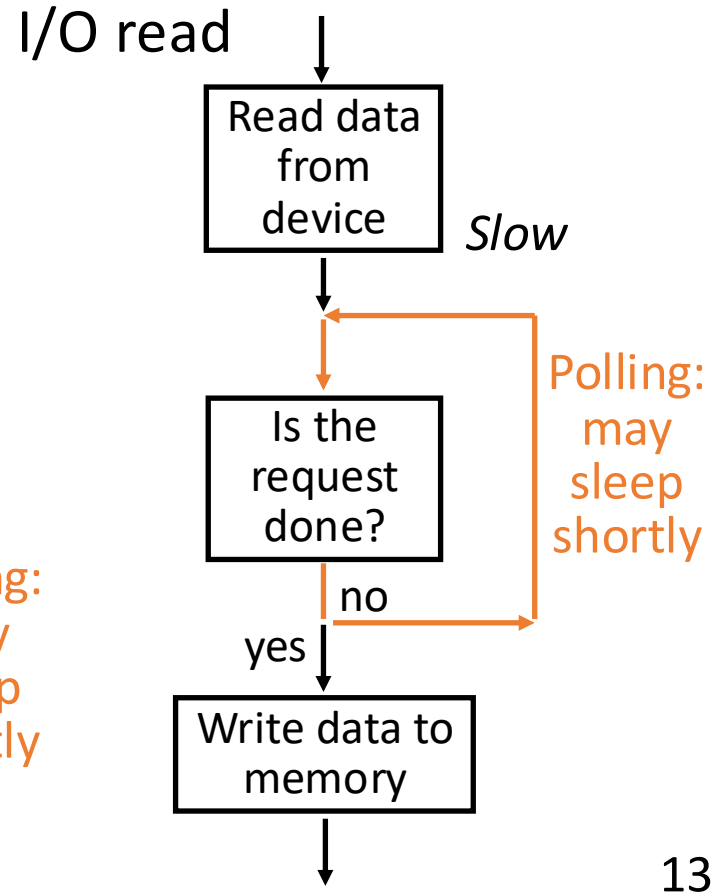
- How does the OS and user programs know that something interesting happened in the I/O device?
  - E.g., an asynchronous event has completed
  - E.g., a packet arrived on the network interface
  - E.g., an error occurred while reading the disk
  - E.g., a key is pressed by the user
- Difficulty: events are infrequent, and occur at any time unexpectedly
- Typical approaches: polling vs. interrupts

# I/O Notification with Polling

- The I/O device places information in a status register
- The OS periodically checks the status register (may be memory-mapped)

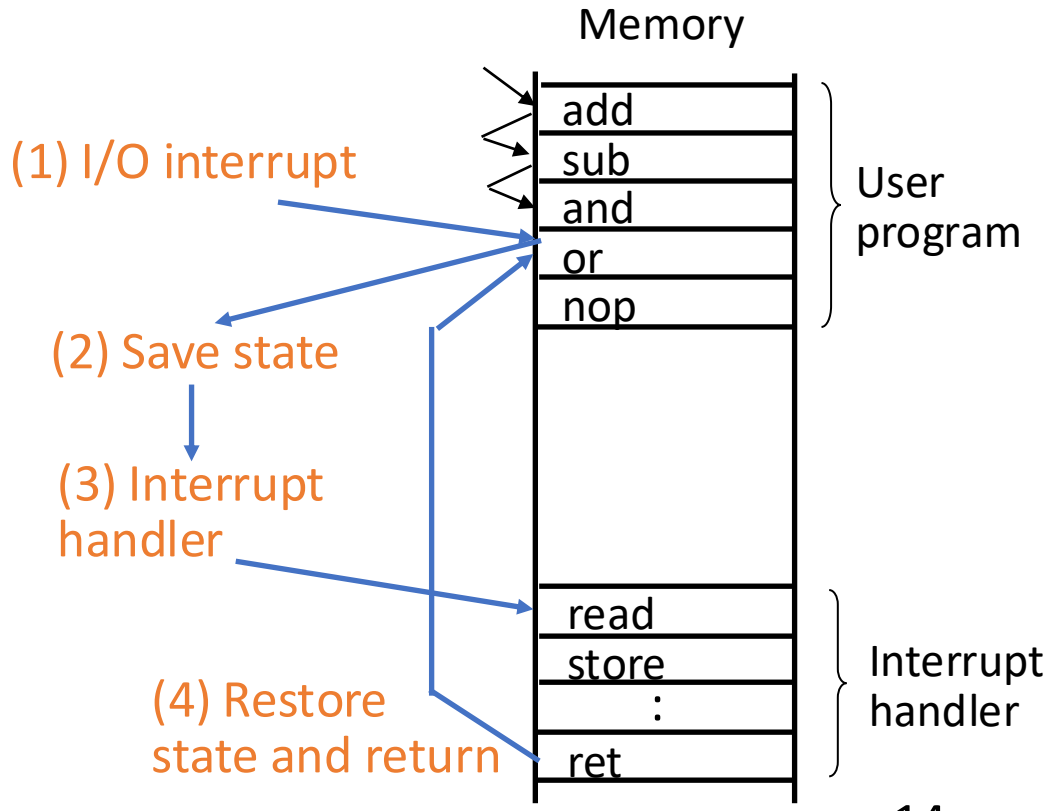
- Advantages
  - Simple to implement
  - Processor is in control and manages the work
- Disadvantages
  - Difficult to determine how frequently to poll
  - Too frequently: waste CPU time
  - Too infrequently: extra delay



# I/O Notification with Interrupts

- When I/O device needs attention, it interrupts the processor
  - Just like an exception but due to an external source, and asynchronously
- The OS finds out the reason from specific places
- Interrupts may have different priorities
  - E.g., network = high priority,
    keyboard = low priority (Q: Why?)
- Advantages
  - CPU is free to do other computation
- Disadvantages
  - Software overhead (e.g., context switch)
  - Implementation complexity



# Polling vs. Interrupts

- Avoid waste CPU time: interrupts is better than polling
- Minimize extra overheads: polling is better than interrupts

- Polling works best for frequent and predictable events
  - The device has a fixed I/O rate so that we know when to poll
  - The device needs frequent attentions so that the interrupt rate would be too high and the interrupt latency is unacceptable
- Interrupts work best for infrequent and unpredictable events
  - The device I/O rate or latency is unpredictable
  - The device can initiate their own I/O events (e.g., raise errors)
  - The CPU is busy with other work
- Q: How to combine their benefits?

# Transferring Data with Programmed I/O

- So far, CPU manages all data transfers: programmed I/O
  - Each byte is transferred via processor through loads/stores between physical addresses of DRAM and I/O devices
  - Pros: simple hardware, easy to program
  - Cons: waste CPU cycles, especially for large data transfers with slow I/O devices

- Want a solution to allow data transfer to happen without CPU's involvement
  - So CPU can focus on computation rather than data transfers, improving overall processing performance

# Direct Memory Access (DMA)

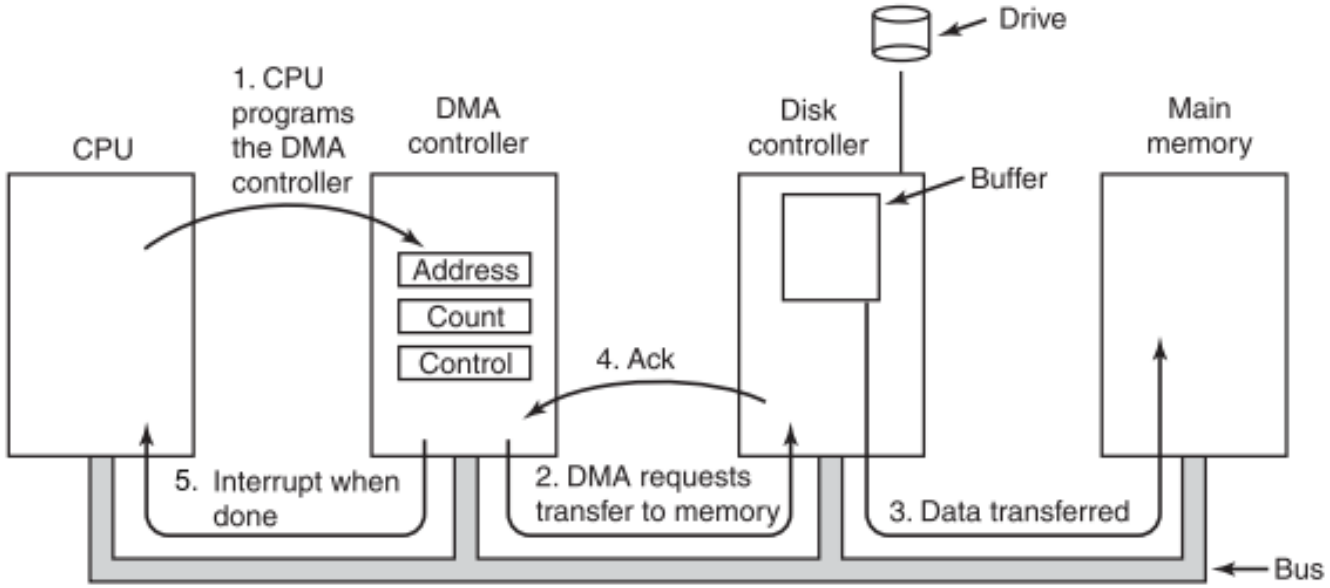
- A custom hardware engine for data movement
  - Transfer blocks of data to or from memory without CPU intervention
  - CPU sends a data transfer task to it, and gets notified when task is done

- DMA engine itself is an I/O device, e.g., with memory-mapped interface
  - Status and control
  - Accept data transfer descriptor from processor: source, destination, length
  - Can keep multiple transfer descriptors in a queue, and serve them one after one

- DMA engines are available in processor chips and I/O chips

# DMA Operations

- Processor sets up DMA by supplying
  - Identity of the device and the operation (read/write)
  - Memory address for source/destination
  - Number of bytes to transfer
- DMA engine starts transfer when data is ready
- Notify processor when complete or on error, typically with interrupts
- Q: Why interrupt, not polling?
- Q: How about using DMA for small amounts of data transfer?



# DMA Design Issues: Virtual Memory

- If DMA uses physical addresses, …

- Issue 1: OS may swap pages out to disk
  - Solution: memory pinning
  - For DMA with physical addresses, pages must be pinned in DRAM, i.e., OS should not page out those pages involved with pending I/O

- Issue 2: contiguous virtual addresses may not be physically contiguous!
  - Solution 1: chain series of single-page requests; single interrupt at the end
  - Solution 2: DMA engine uses virtual addresses
    - Then those virtual addresses need to be translated; by whom and how?

# DMA Design Issues: Cache Coherence

- Issue 3: data involved in DMA may reside in processor cache
  - If written by I/O devices: must update or invalidate staled cached copies
  - If read by I/O: must read latest values, maybe from the cache
  - Another version of the cache coherence problem

- Solution 1: software flushes the cache or forces writebacks before I/O
  - Be selective or for the whole cache
  - Overheads may be high
  - Need hardware (instruction) support
- Solution 2: hardware routes memory accesses for I/O through cache
  - First search the cache for copies and invalidate or write back as needed
  - May impact performance negatively
    - While searching cache for I/O requests, it is not available to processor

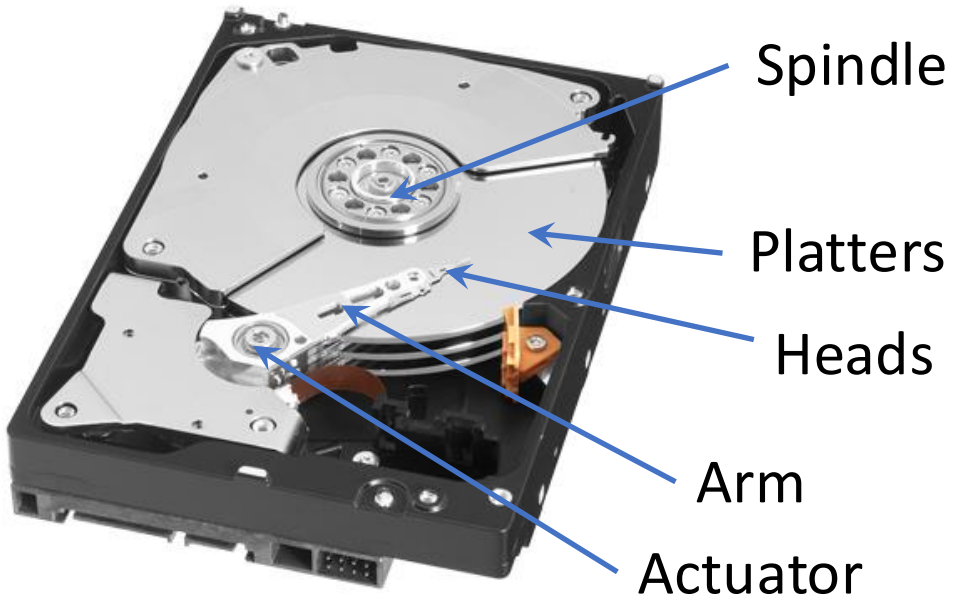
# Summary

- I/O devices
  - Different speeds: 0.1 byte/s to GB/s
  - Different access patterns: block, character, network devices
  - Different access timing: blocking, non-blocking, asynchronous
- Memory-mapped I/O
  - I/O devices are mapped to physical address space
  - User processes access I/O through syscalls or virtual memory
- Device driver: device-specific code in kernel to expose uniform interface
- I/O notification: polling vs. interrupts
- DMA for efficient large data transfers
  - Issues of virtual address, cache coherence

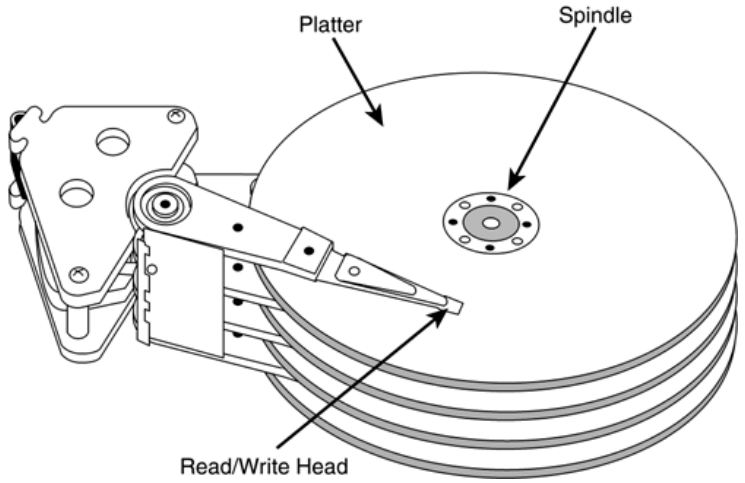
# I/O Devices: Disks and SSDs

# Magnetic Hard Disks

- Long-term, non-volatile: magnetic
- Large, inexpensive
  - Capacity continuously increases every year
  - Price below $0.1/GB
- Slow: mechanically rotating
  - Moving heads access the disk

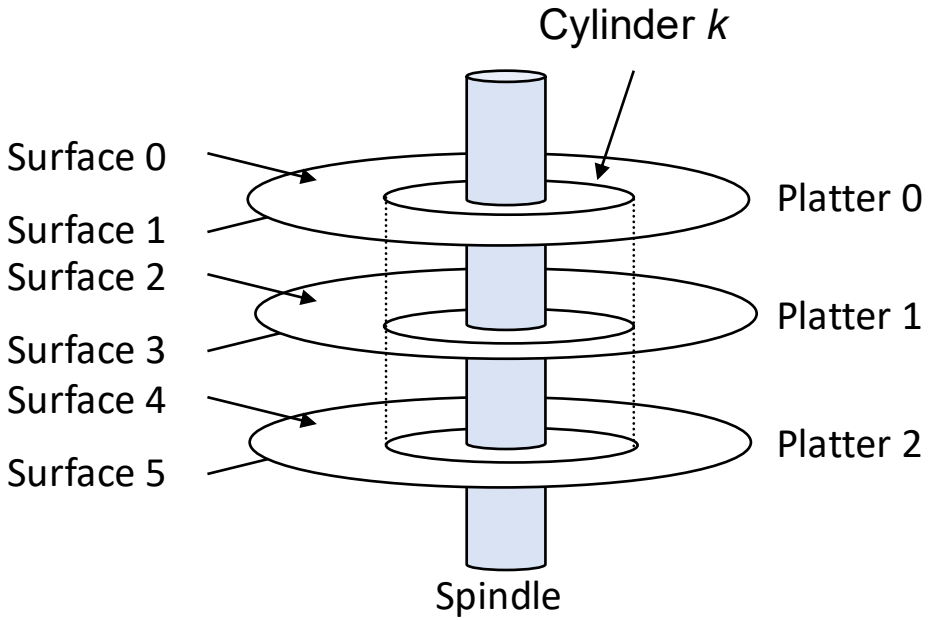



- Usage
  - Virtual memory (swap area)
  - File system (next lecture)

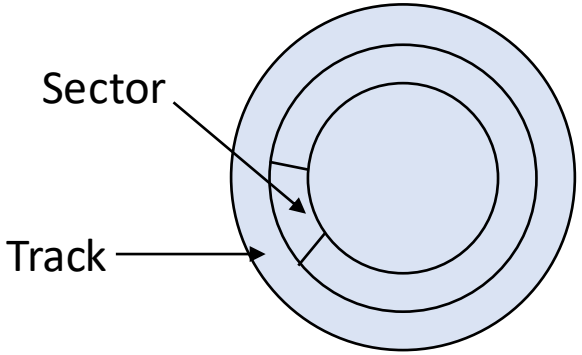


# Hard Disk Organization

- Each disk consists of platters
- Each platter has two surfaces
- Each surface consists of tracks (rings)
- Each track consists of sectors
  - Sector records data, e.g., 512 B/sector
- Aligned tracks form a cylinder

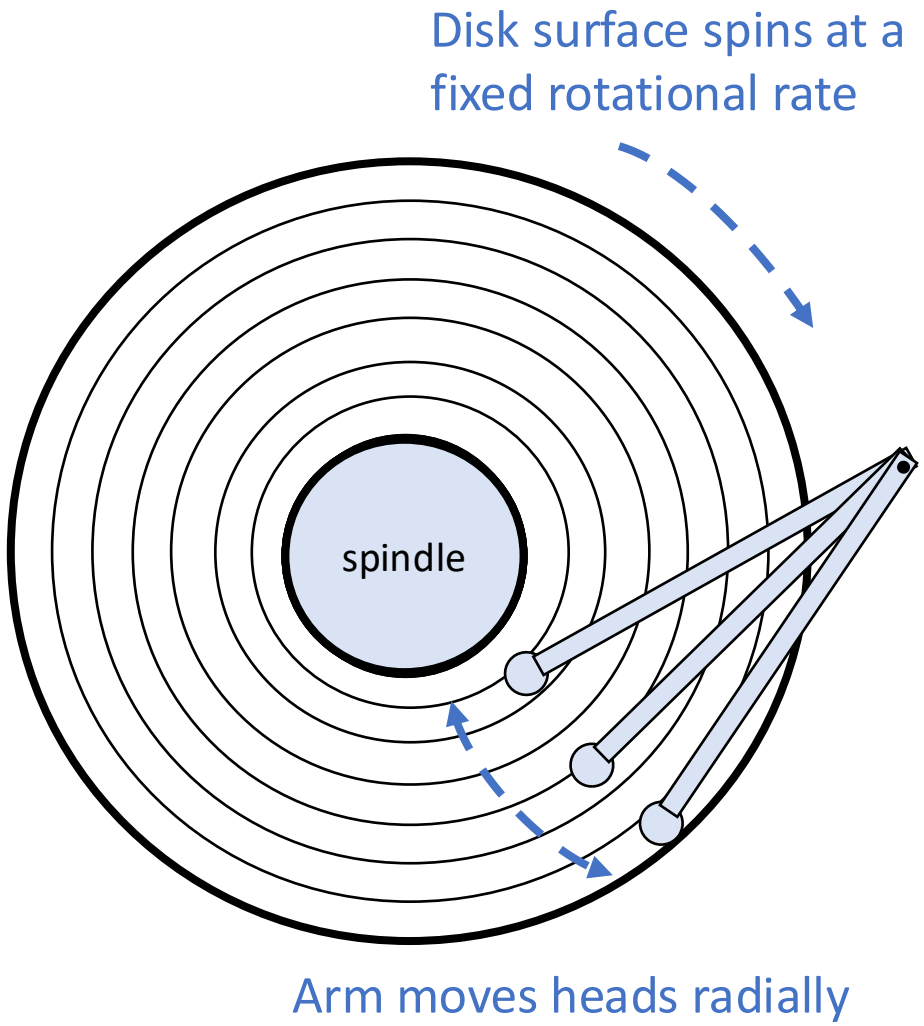



- Software always transfers groups of sectors together: blocks
  - E.g., from the sectors of a cylinder

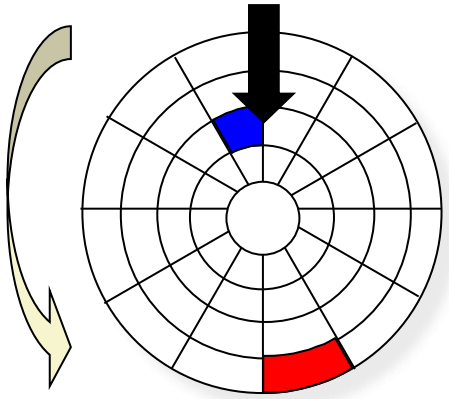


# Disk Access Time Breakdown

- Controller overheads
- Queuing delay if other accesses are pending
- Seek time: move heads to the track
  - Typically a few milliseconds (slow!)
- Rotational latency: wait for target sector in the track
  - Rotational rate: rpm (rounds per minute)
  - Average: rotate half circle
- Data transfer time: the whole sector passes under the heads
  - All heads read a cylinder (all surfaces) together

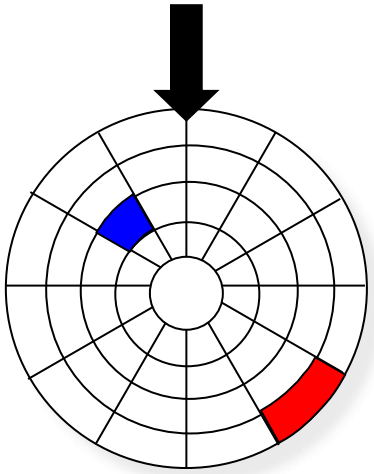


# Disk Access



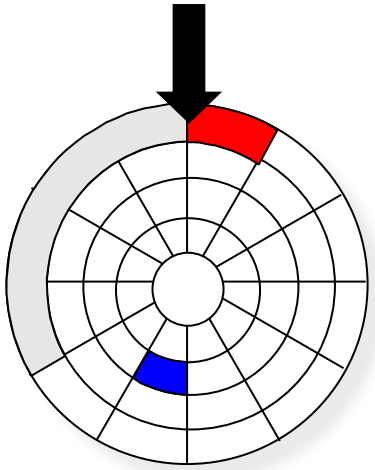
After BLUE read

Data transfer



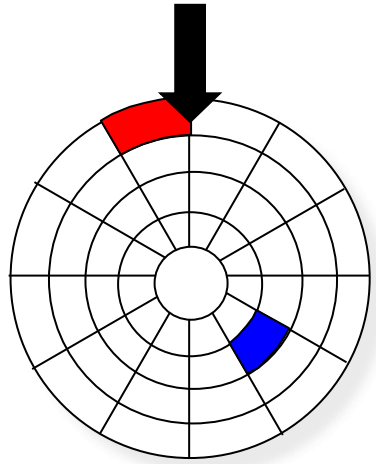
Seek for RED

Seek



Rotational latency

Rotation



After RED read

Data transfer

# Disk Performance Example

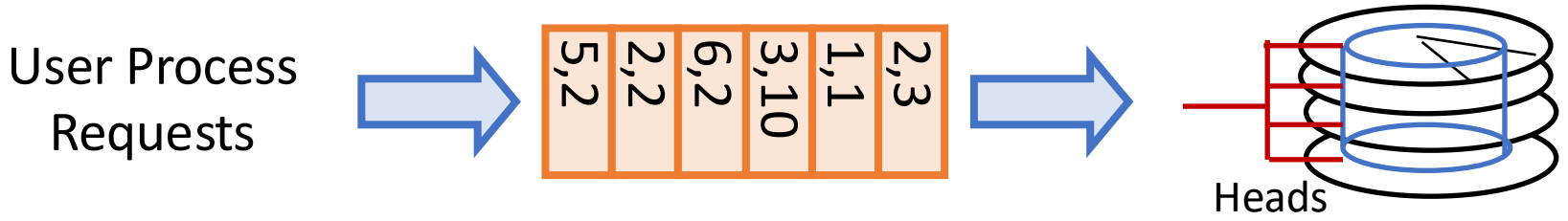
- 4 kB block, 15,000 rpm, 4 ms average seek time, 100 MB/s transfer rate, 0.2 ms controller overhead, idle disk
  - Average read time =
    4 ms seek time
    + ½ / (15,000 / 60 sec) = 2 ms rotational latency
    + 4 kB / 100 MB/s = 0.04 ms transfer time
    + 0.2 ms controller overhead
    = 6.24 ms
  - Throughput = 4 kB / 6.24 ms = 656.4 kB/s
- If read 1 MB data *sequentially*
  - Average read time = 4 ms + 2 ms + 1 MB / 100 MB/s + 0.2 ms = 16.2 ms
  - Throughput = 61.7 MB/s, 100x improvement!

# Disk Performance Issues

- Access to small data dominated by seek time and rotational latency
  - SRAM access time is about  1 - 4 ns, 1,000,000 times faster than disk
- Q: What optimizations we have learned so far could help?
- Disk scheduling that exploits data locality can improve performance
  - Large sequential accesses well amortize seek and rotational overheads
  - If consecutively accessed data are close, seek and rotation time can be reduced
- Disks can also use caches
  - Avoid seek and rotational delays

# Disk Scheduling

- When there are several queued requests to a disk, in what order should we serve them?
- Example: requests are denoted as (track ID, sector ID)




- We simplify the problem by just considering track IDs
  - Seek time dominates; rotational latency is smaller than seek time

# Disk Scheduling Policies

- Requests: 2, 1, 3, 6, 2, 5
- Initial head position: track 5

- FIFO: in the order of arrival
  - Scheduling order: 2, 1, 3, 6, 2, 5
  - Pros: fair among requests
  - Cons: many seeks

- Shortest seek time first (SSTF): pick the request that is closest to head
  - Scheduling order: 5, 6, 3, 2, 2, 1
  - Pros: reduce seeks
  - Cons: may lead to starvation

# Disk Scheduling Policies

- Requests: 2, 1, 3, 6, 2, 5
- Initial head position: track 5

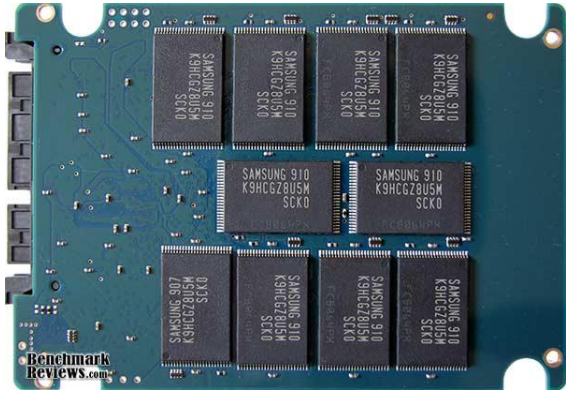
- SCAN: elevator algorithm, take the closest request in the travel direction
  - Scheduling order: 5, 6, 3, 2, 2, 1
  - Pros: low seek, no starvation
  - Cons: still unfair as it favors middle tracks

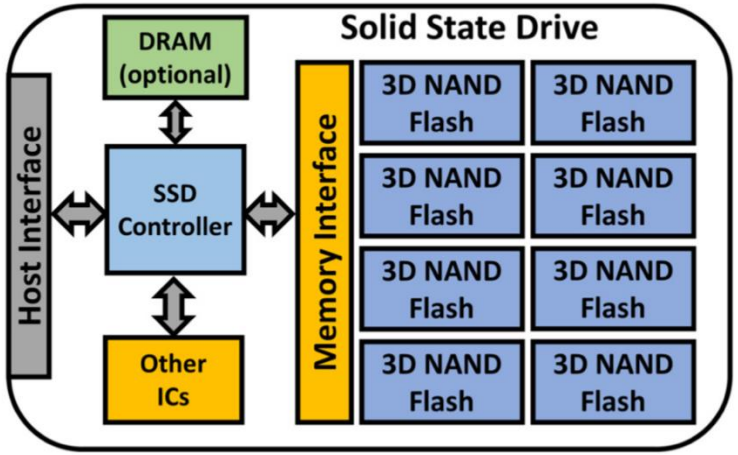
- C-SCAN (circular SCAN): like SCAN but only serves in one direction
  - Scheduling order: 5, 6, 1, 2, 2, 3 (assume serve direction is from center to edge)
  - Pros: more fair than SCAN; more consistent response time
  - Cons: longer seeks on the way back; lower throughput

# Solid State Drive (SSD), a.k.a., Flash

- SSDs are usually made by NAND/NOR multi-level cell Flash memory
  - An array of Flash chips
  - A DRAM buffer (a.k.a., cache)
  - A controller, which could be either a general-purpose core or specialized chip
- No slow mechanical moving parts
  - Seek and rotational latencies are eliminated
  - Lower total power, more rugged, and lightweight

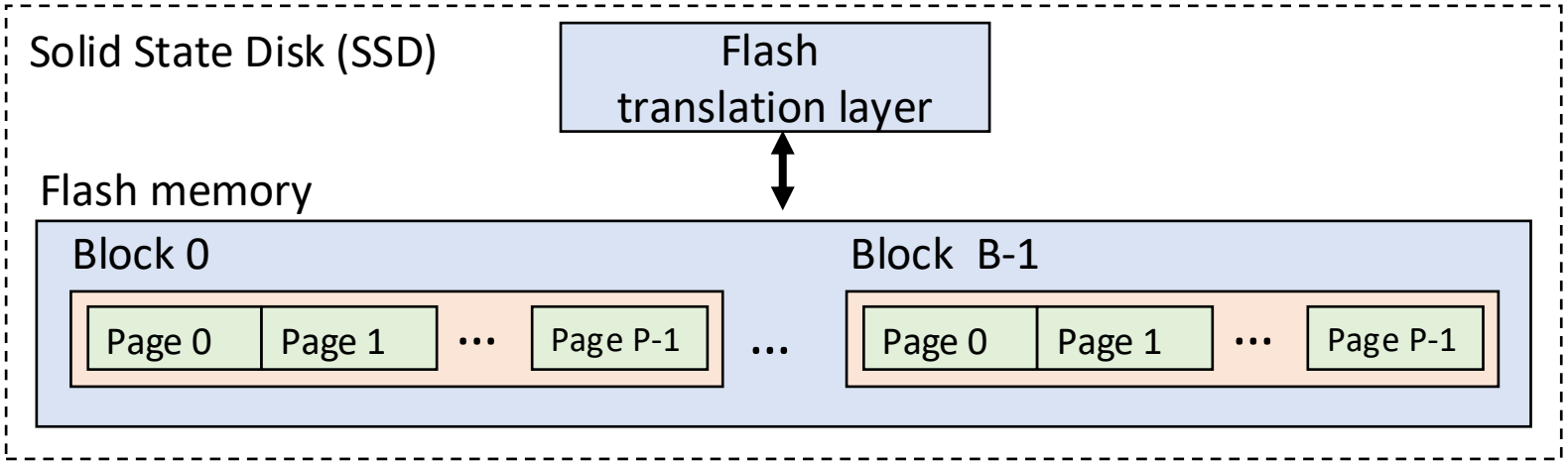


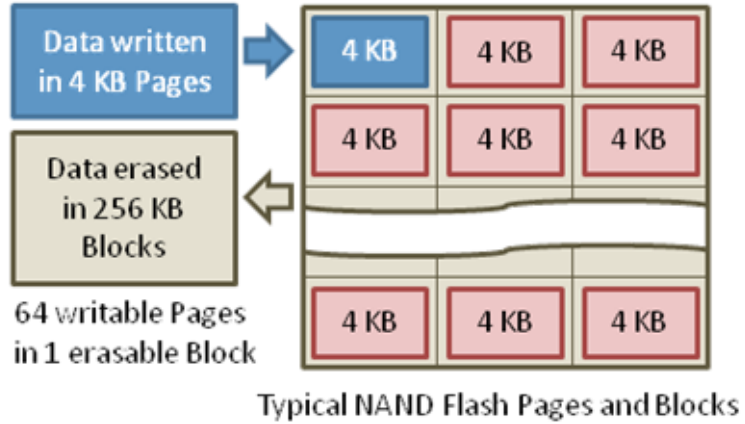


# SSD Structures and Operations

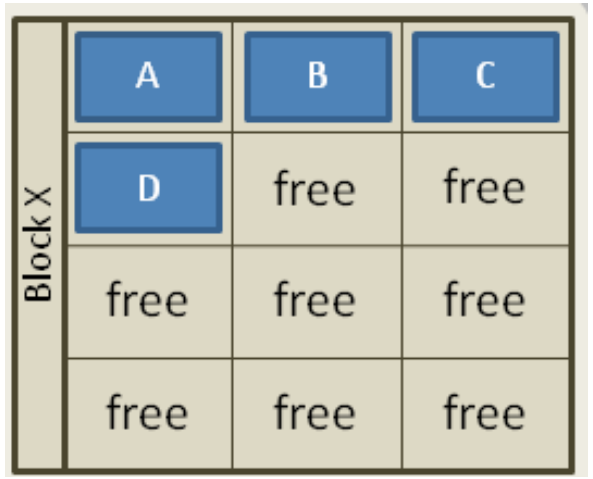
- 4 kB per page; 32 to 128 pages per block
  - Two levels due to technology limitations below
- Reads are in units of pages, with latency in about 20 us
  - Similar to blocks in disks, but without long seek and rotation
- Writes are complicated, with latency in 200 us – 1.7 ms
  - Can only write to erased pages (about 200 us)
  - Erase must happen in units of blocks and are slow (e.g., 1.5 ms)



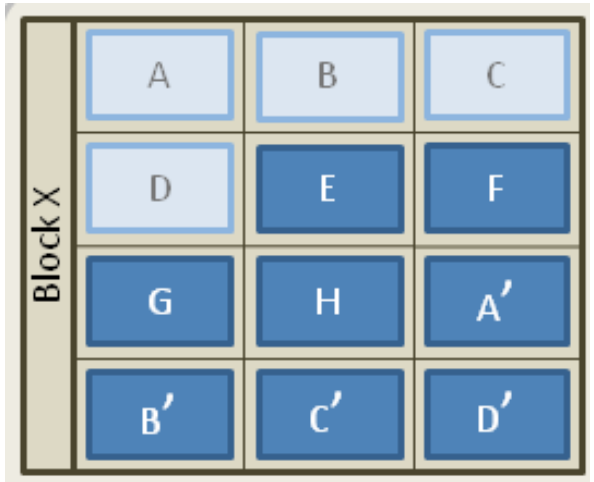



Typical NAND Flash Pages and Blocks

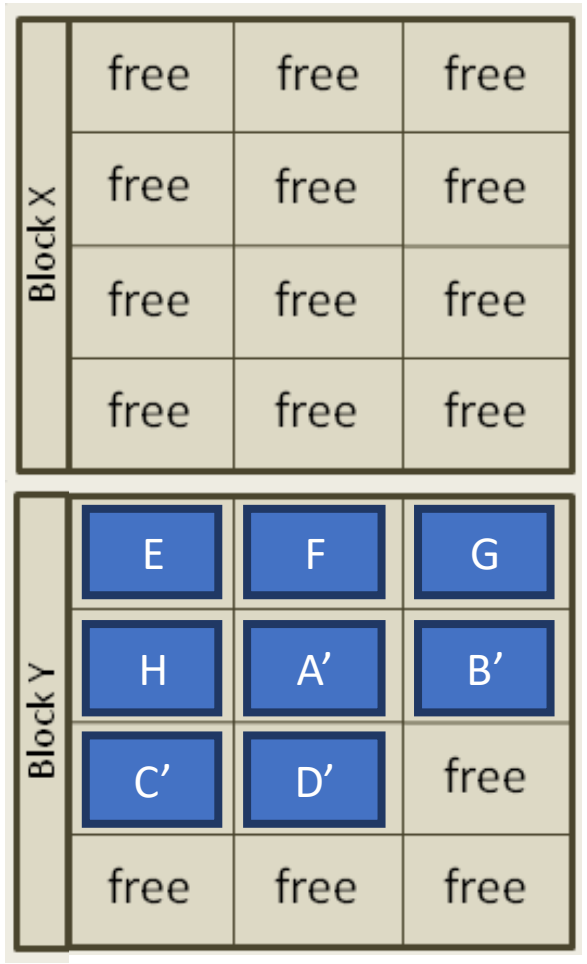
# Example of SSD Operations




- Start with an empty block X
- Write pages A, B, C, D




- Write pages E, F, G, H
- Modify A, B, C, D by writing new A', B', C', D'
- Change address map from A, B, C, D to A', B', C', D'
- Mark A, B, C, D as obsolete




- Controller garbage collects obsolete pages by copying valid pages to a new block Y, and then erase block X

# Flash Translation Layer (FTL)

- No in-place modification in SSD/Flash. Must write data to a new page in an erased block; then let the address point to this page

- Such management is handled by the Flash Translation Layer (FTL)
  - Typically a software layer running on the internal controller
- FTL functionalities
  - Data address mapping
  - Tracking physical block status
  - Wear leveling

# Main Data Structures in FTL

- Address mapping: logical to physical
  - Similar to virtual memory
- Block info table: for each physical block
  - Track block status
    - Erased or not, when to garbage collect
  - Write pages in a block sequentially
    - By recording the last written page (i.e., sequence number) of each block
  - Wear leveling

Address Map

| Logical Addr | Physical Page Address | |
|---|---|---|
| 0 | Block 5 | Page 7 |
| 4k | Block 27 | Page 0 |
| 8k | Block 10 | Page 2 |

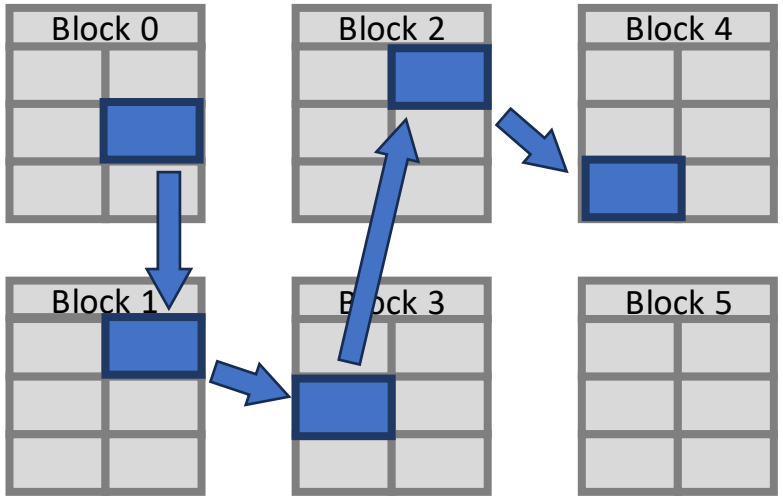
Block Info Table

| Block | Erased | Erase Count | Valid Page Count | Valid Page Bitmap | Sequence Number | Bad? |
|---|---|---|---|---|---|---|
| 0 | False | 3 | 5 | 00110…0 | 15 | False |
| 1 | True | 7 | 0 | 00000…0 | 0 | False |
| 2 | False | 0 | 4 | 11110…0 | 9 | False |

# Wear Out and Wear Leveling

- SSD has limited lifetime during which we can reliably store data
- A block wears out after certain number of erases, e.g., 100,000
  - Write and erase require high voltage that gradually damages cells
- Frequently written data may wear out their pages/blocks sooner

- FTL performs wear leveling to avoid hotspots
  - To uniformly move and remap data to different physical pages/blocks
  - Happen at the time of write and erase when choosing new physical locations



# SSD Summary

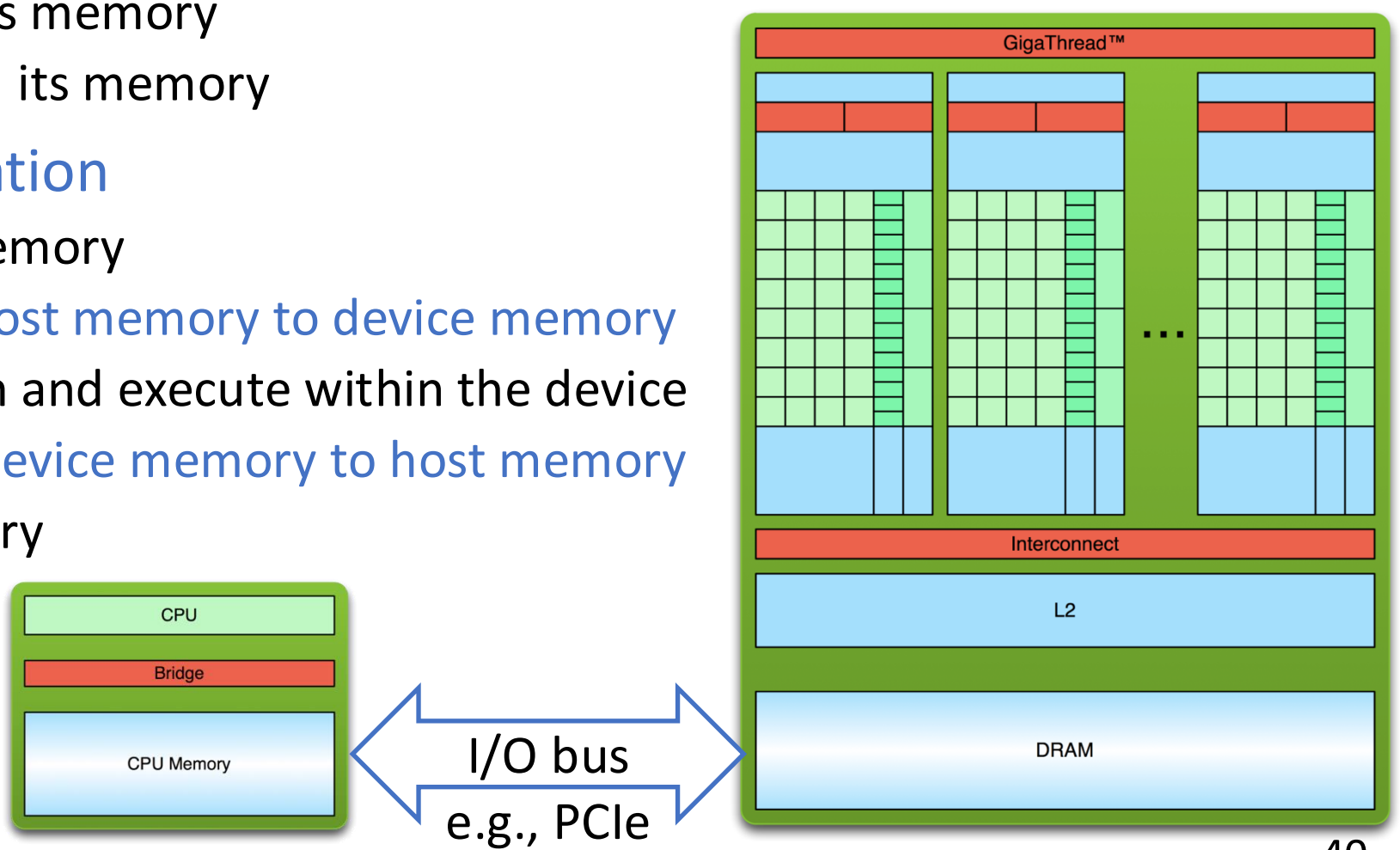
- Advantages (vs. disks)
  - No moving parts → faster, less power, more rugged
  - Read latency: 20 – 100 us, vs. a few ms of disks
  - 5-10x higher bandwidth than disks, but still >10x lower bandwidth than DRAM
  - Random access is only slightly slower than sequential access

- Disadvantages
  - Asymmetric performance between reads and writes: writes are ~10x more expensive than reads, and erases are ~10x more expensive than writes
    - Managed by page-block structure and FTL
  - Have the potential issue of wear-out; mitigated by wear leveling in FTL
  - Still more expensive per byte than disks

# I/O Devices: GPUs

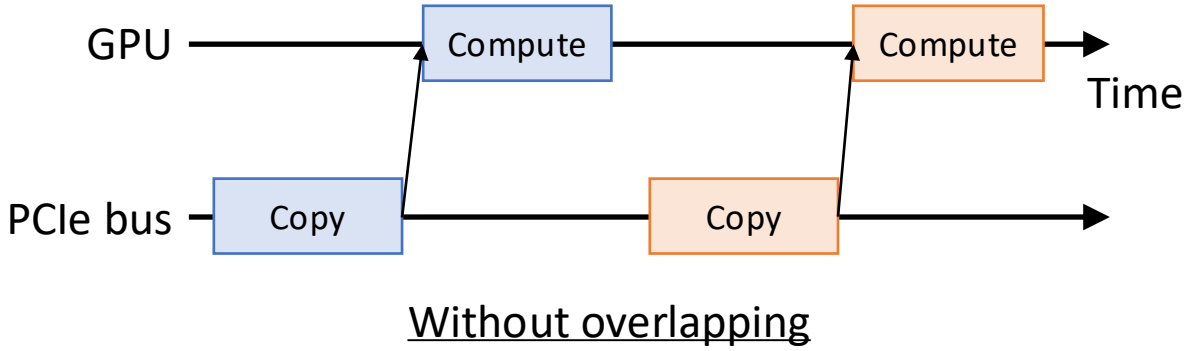
# GPU in System

- The GPU is connected as an I/O device to the CPU
  - Host: the CPU and its memory
  - Device: the GPU and its memory
- Offloading computation
  1. Allocate device memory
  2. Copy input from host memory to device memory
  3. Load GPU program and execute within the device
  4. Copy result from device memory to host memory
  5. Free device memory

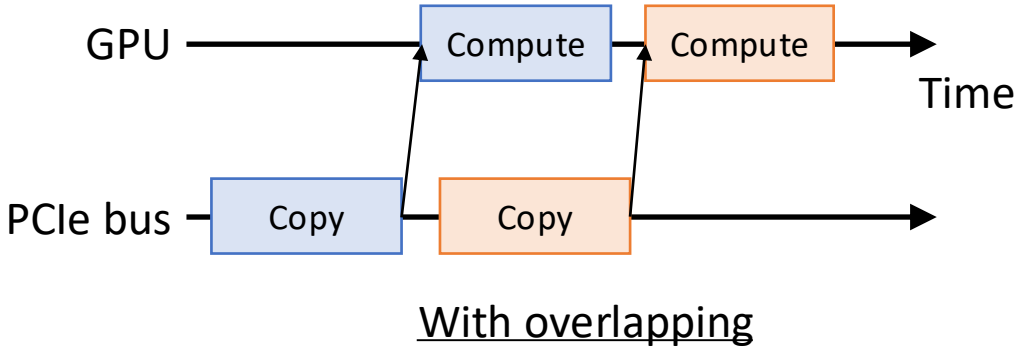


# Communication between Host and GPU

- CPU and GPU have physically separate memories
- Performance overhead of data movements is significant
  - Hundreds to thousands of cycles latency, 1/10 to 1/100 of on-chip bandwidth
- Optimization: overlapping computation and communication




Without overlapping




With overlapping

# Host & GPU

- As management processor + accelerator
  - CPU is dedicated to GPU management and other auxiliary functions
    - E.g., network I/O, disk I/O, system memory management, scheduling decisions
  - CPU can block, or work on other things
    - Allow for a light-weight, inexpensive CPU to be used
  - GPU is the workhorse of the application, offloaded with most computation

- As cooperative co-processors, i.e., heterogeneous computing
  - CPU and GPU work together on a problem
  - Keep both processors well-utilized, i.e., fitting with their characteristics
    - Parallelism vs. single-thread speed, data exchange, …
  - Require good work partitioning techniques across CPU and GPU

# Next Lecture:
# File Systems and Protection